

XPression Hands-on Practice Handbook

DekoBokoTech / XPression Field Notes

This handbook teaches practical XPression work hands-on, using the bundled sample assets. Each exercise has a matching folder of materials.

* All sample assets are original dummies — no real teams, logos, or colors are used.

Included Hands-on Exercises

No.	Title	Level	Time	Folder
1	Build an Unbreakable Lower Third	Intermediate	30 min	01_LowerThird
2	Build a Scoreboard	Intermediate-Advanced	40 min	02_Scoreboard
3	Swap Images with DataLinq	Intermediate	25 min	03_DataLinq_ImageSwap
4	Build a Simple Game Clock	Intermediate	30 min	04_GameClock
5	Feed Scores and Names from CSV / Excel	Intermediate	35 min	05_DataLinq_Practice
6	Headline Reveals One Character at a Time with Stagger	Intermediate	25 min	06_Stagger_Text
7	Design Ins and Outs with Transition	Intermediate	25 min	07_Transition
8	Build a Staff Roll (Roll)	Intermediate	30 min	08_StaffRoll
9	Create a Visible Area with Mask	Intermediate	25 min	09_Mask
10	Build Readable Color Fields with Gradient	Beginner	20 min	10_Gradient
11	Show the Current Time and a Countdown	Intermediate	30 min	11_Clock_Countdown
12	Build a Virtual Monitor with Render View	Intermediate	30 min	no materials needed (built with features only)
13	Cycle a Player List with JSON	Intermediate	40 min	13_JSON_PlayerList
14	Take from Outside with RossTalk	Intermediate	30 min	14_RossTalk

The materials for each exercise are inside its folder. Work through the steps using those files.

1. Build an Unbreakable Lower Third

Splitting work between native features and Visual Logic

Vol. 4, Ch. 12 | Level: Intermediate | approx. 30 min | Folder: 01_LowerThird

Goal

A two-line lower third with name and title. The background hugs the text nicely whether the name is short or long, with a hard cap so it never leaves the safe area. When the title is empty, the row disappears and the name alone looks clean. It shows and hides reliably with Online / Offline / Take.

STEP 1 Build the container (background parented to the Text Object)

The container here is "name Text + its child background" plus "title Text".

- ① Drag a Text from the Object Library into the Scene, make the name Text, and rename it Name_TXT in the Object Manager
- ② Add a background object (Quad or Background) and drag it in the Object Manager so it becomes the child (First Child) of Name_TXT (this parent-child link is what makes Auto Scale work later)
- ③ Add a title Text as Title_TXT. Enter a placeholder name and title
- ④ Keep the background color in a Material (holding color as a value makes team-color swaps easy later)

Check: In the Object Manager the background hangs under Name_TXT, and each name alone tells you the role.

STEP 2 Build the width foundation natively (this is the main event)

On the name Text Object's Tabs & Options tab, set three things.

- ① Auto Squeeze: Enabled, enter the max width in Max Width, Scaling = Width Only (squeeze horizontally only).
- ② Auto Scale: Enabled, Target = First Child, Mode = Width & Height — the child background follows the parent's squeeze (no need to compute background width in Visual Logic).
- ③ Character Limits: Soft Limit warns (text turns red), Hard Limit caps the character count (the final insurance).

Width handling is now done — no Visual Logic at all.

Check: A long placeholder name squeezes horizontally and never exceeds Max Width. The background follows. Text turns red past the Soft Limit.

Tip: The exact Max Width / Hard Limit values depend on the safe area and the font. Decide them on the real system by entering the longest name.

STEP 3 Visual Logic, part 1: if the title is empty, hide the row

Build "hide the whole row when the title is empty" in Visual Logic.

- ① Select the Scene and open Visual Logic (Visual Logic Editor)
- ② Expand Title_TXT under Objects on the left and drag Text (the value) onto the canvas
- ③ From Function Blocks drag String Length (Strings) and Greater Than (Logic)
- ④ Wire: Title_TXT.Text → String Length.In / String Length.Out → Greater Than.Base (threshold 0 = "has at least one character?")
- ⑤ Wire Greater Than's output to the Visible of the title row (the Group holding Title_TXT and its background)

Empty gives 0 → hidden; non-empty gives 1 → shown. To recenter the name, wire the same result to the name Text's Position(Y) too.

Check: Clearing the title removes the title row; entering one brings it back.

STEP 4 The design decision: switch just before it becomes unreadable (Visual Logic, part 2)

Build the machine-verified setup as-is:

- ① In Visual Logic wire Name_TXT's Text → String Length (Strings)
- ② Place a Value (Math) and enter the threshold character count (verified example: 43)
- ③ String Length.Out → Greater Than.Base (Logic), Value → the threshold side
- ④ Greater Than output → Name_TXT's WordWrap.Enabled (wrap turns on past the threshold)
- ⑤ String Value (Strings, verified example 1520) → WordWrap.Width to fix the wrap width

The wrapping itself is native Word Wrap; Visual Logic only decides when to switch.

Check: Only names past the threshold wrap to two lines and stay readable.

Tip: Approximate the switch threshold by character count. It depends on resolution, safe zone, and font — decide on the real system.

STEP 5 Tidy up show and hide

- ① Drag the Scene into the Sequencer to make a Take Item, swap the published name/title via Template Links, and prepare four Take Items: "short name", "long name", "over the limit", "no title"
- ② Play out with Take (numpad +, TAKE via RossTalk), remove with Offline (numpad —, SEQO)
- ③ Run all four patterns in rehearsal and confirm nothing breaks and everything reads
- ④ After a swap: check preview → re-TAKE to update the output (iron rule)

Check: In every pattern the background follows and nothing leaves the safe area.

Pre-show checklist

- ☐ With the longest expected name, Auto Squeeze kicks in and nothing leaves the safe area

- ☐ The background follows the name via Auto Scale without broken margins
- ☐ With an absurdly long name, Hard Limit works as the final insurance
- ☐ The switch happens before over-squeezing makes text unreadable (fits $\neq$ reads)
- ☐ With no title, the name alone sits cleanly without gaps
- ☐ Swapping the name during playout doesn't break the squeeze or background tracking

2. Build a Scoreboard

Where Visual Logic shines

Vol. 4, Ch. 13 | Level: Intermediate–Advanced | approx. 40 min | Folder: 02_Scoreboard

Goal

A two-team scoreboard. Team names, colors, scores, and period for home and away. A lead highlight that shows at a glance who's winning. Changed digits get a light emphasis when the score moves, and entrances/exits are smooth.

STEP 1 Build the container (scoreboard object structure)

- ① In a new Scene, place Text ×4 (HomeName_TXT / HomeScore_TXT / AwayName_TXT / AwayScore_TXT), a background panel, and lead markers (HomeLead / AwayLead: glow or arrows), mirrored left-right
- ② Place Period_TXT for the period and Timer_TXT for the clock
- ③ In the Object Manager unify names so side and role are obvious (an investment so you don't get lost wiring later)

Check: Home/Away parts are mirrored and each name alone tells you the role.

STEP 2 Shape scores for display

Machine-verified setup (NumFormat tab):

- ① Add a tab in the Visual Logic Editor and name it NumFormat (splitting tabs by function keeps it readable later)
- ② Place two Format String blocks (Strings)
- ③ Wire: raw score value → Format String.Param → SB_HomeScore_TXT.Text (same for Away)
- ④ Zero-padding like "07" is decided by the Format pattern. Never show raw numbers — shape them right before display

Check: 1-digit and 3-digit scores both display as intended.

Tip: Decide the expected digits (max score) and zero-padding per sport and operation, and check digit-overflow behavior on the real system.

STEP 3 Visual Logic, part 1: highlight the lead

- ① Compare home and away scores with Greater Than (Logic)
- ② Wire the result to HomeLead / AwayLead Visible (only the winning side shows)
- ③ Build the tie branch from the start: detect ties with Equal To and hide both markers (or switch to a tie display). Left alone, it looks like both teams are winning
- ④ For color intensity too, route the result through Colors blocks to the Material

Check: All three states — home lead, away lead, tie — highlight correctly.

Tip: For the exact Function Block wiring of comparison, color switching, and tie detection, treat what you build on the real system as the source of truth.

STEP 4 Visual Logic, part 2: switch team colors

- ① Take the team selection (a Widget Text List or published Text) as input and pick the matching team color with Input Selector (Selectors)
 - ② Wire the chosen color to the background/marker Material (Colors.Diffuse)
- Visual Logic decides which color; native (Material) displays it. That's the division of labor.

Check: Switching the team selection changes background and marker colors to that team's colors.

STEP 5 Add motion natively

Machine-verified setup:

- ① Build In/Out moves in the Scene Director and combine with the Sequencer's Transition In/Out columns. Order is managed in the Sequencer
- ② For score emphasis, build ScoreMove (a short bounce of the digits) in the Scene Director and park it with an Event (Stop)
- ③ Add a PointMove tab in Visual Logic and wire SB_HomeScore_TXT's Events.OnSetText → Scene Director (ScoreMove) Actions.Play (same for Away)

The emphasis auto-plays the instant text is replaced. Always verify the display survives the moment values move.

Check: Changing the score triggers the emphasis and nothing breaks.

STEP 6 Advanced: a bar proportional to the score (optional)

Machine-verified setup (%BAR tab):

- ① HomeScore and AwayScore → Add (Math) (the total)
 - ② HomeScore → Divide.Base, Add.Out → Divide.Divisor (home share, 0–1)
 - ③ Guard against 0-0: Add.Out → Equal To (==0) → Input Selector choosing "0.5 if total is 0, else the ratio" (the standard division-by-zero guard)
 - ④ Chosen value → Multiply ($\times$ max bar width, verified example 1500) → HomeBar_SHAPE.Width
- The ratio is 0–1, so the max width caps it naturally.

Check: Even lopsided scores stop the bar at the max width. 0-0 shows half and half.

Pre-show checklist

- ☐ All three states — home lead, away lead, tie — highlight correctly
- ☐ At a tie, it doesn't look like both teams are winning
- ☐ Swapping teams switches colors correctly
- ☐ Scores going from 2 to 3 digits stay readable without overflow
- ☐ Nothing breaks the instant scores swap or tie
- ☐ Entrances and exits are smooth and playout order stays intact

3. Swap Images with DataLinq

Dynamic Material and Material Path (auto-swapping images and logos)

Vol. 5, Ch. 3 | Level: Intermediate | approx. 25 min | Folder: 03_DataLinq_ImageSwap

Goal

Build a template that switches team logos, player photos, and sponsor assets without pasting images by hand each time. The displayed image is decided by its file location (path), and changing a Widget or data value swaps in a different image. When the matchup or lineup changes, there's no manual repasting.

STEP 1 Organize the asset files

First, organize the images you'll swap in a place that's easy to reference later. Create an "Images" folder inside the project folder, split subfolders by purpose, and prepare files like:

[Example folders and files]

Images/Teams/

- TokyoTeamLogo.png

- OsakaTeamLogo.png

- NagoyaTeamLogo.png

Images/Players/

- Player_01_Photo.png

- Player_02_Photo.png

Images/Sponsors/

- Sponsor_Main.png

Images/Backgrounds/

- Arena_Background.png

Key point: unify team logo filenames as "team name + Logo.png" (TokyoTeamLogo.png / OsakaTeamLogo.png ...). Then in step ⑤, changing the Widget value (TokyoTeam / OsakaTeam ...) automatically switches to the matching file.

Check: Folders like Images/Teams contain files following the naming rule.

Tip: Keep folder and file names in single-byte alphanumerics to avoid path trouble. Decide the file-naming scheme (value + fixed word) first, and the automatic switching in step 5 falls right into place.

STEP 2 Prepare the object that displays the image

Place an object to show the image.

① Drag Primitives > Quad from the Object Library into the Scene (a Quad works well for team logos and sponsor images)

② Name it by role in the Object Manager, e.g. "HomeLogo"

If you already have a logo frame, use that.

Check: A Quad is placed with a role-revealing name like HomeLogo.

Tip: Think of it as preparing the surface the image is pasted on. A fixed image is fine for now (we make it dynamic next).

STEP 3 Switch the material to Dynamic Material

Select the object, open the Object Inspector, go to the "DataLinq" tab, and set "Select Material Source" to "Dynamic Material". The object's image is now assignable later instead of fixed.

Check: Select Material Source on the DataLinq tab shows Dynamic Material.

Tip: Dynamic Material = a material whose displayed image is decided dynamically by a path or data. This is the heart of image swapping.

STEP 4 Specify the image with Material Path

In Dynamic Material Properties, enter the image file's location (path) into "Material Path". Relative to the project folder is the norm.

Examples:

- Team logo → Images/Teams/TokyoTeamLogo.png
- Player photo → Images/Players/Player_01_Photo.png
- Sponsor → Images/Sponsors/Sponsor_Main.png
- Background → Images/Backgrounds/Arena_Background.png

The file specified here is what the object displays.

[Important: a checkerboard in the Layout view is OK — it displays on the playout side]

In the Layout view, Dynamic Material is not previewed; you see a blue-and-yellow checkerboard placeholder. This is normal and does not mean the path is wrong. The real image displays on the playout side (Sequencer / Take = on air). Check the display via the Sequence first.

[Only if the image is also missing on the Sequence side, re-check the path]

If it stays checkerboard on the Sequence too, the path isn't resolving. Check:

- Are there double quotes around the path? "Copy as path" pastes like "C:/.../TokyoTeamLogo.png" — the quotes must be deleted.
 - Are separators / rather than ¥?
 - Any Japanese or full-width characters in folder/file names? (half-width alphanumerics are safe)
 - Does the file actually exist there, with matching name and extension (including case)?
 - For relative paths: is the project saved, and is the Images folder under the saved project?
- (To position/size in the editor, temporarily assign the same image as a Static Material while working, then switch back to Dynamic Material before playout.)

[How to copy a path (Windows)]

- Shift + right-click the file in Explorer → "Copy as path".
- For just the folder path, click the address bar and copy.

What you copy is an absolute path. For a relative Material Path, keep only the part below the project folder (e.g. Images/Teams/...). "/" separators are fine.

- Important: "Copy as path" may add quotes around the path (e.g. "C:/.../TokyoTeamLogo.png"). Always delete them — if they remain, the path won't resolve and you'll keep the checkerboard.

Check: The logo displays from a hand-typed path in Material Path (if the checkerboard persists, suspect quotes or typos).

Tip: If you use absolute paths (D:/...), always confirm the same path resolves on the production machine. Relative paths survive moving to another PC and are safer.

STEP 5 Switch the filename via a Widget (Text List) — exact syntax

This is the heart of auto-switching. First create the switching Widget.

Display menu → Widgets panel → "New Widget" → "Text List" (a widget whose text you can switch; use Clock Timer for time, Counter for numbers).

Rename "TextList1" to something clear (e.g. TeamNameForFile).

Next, open Widget Properties and register the values with "Add".

Example: TokyoTeam / OsakaTeam / NagoyaTeam ...

※ Checking "Allow manual entry of text" lets you type values not in the list.

Then write the macro @W:widgetname@ into the filename part of the Material Path.

Example:

Images/Teams/@W:TeamNameForFile@Logo.png

[No hand-typing: insert via right-click]

You don't have to type the macro. Right-click in the Material Path field → "Insert Lookup" → "Widgets" → pick your Widget (e.g. TeamNameForFile) and @W:TeamNameForFile@ is inserted automatically.

The same menu offers "Published Text Objects" / "All Text Objects" to insert text-object values.

This avoids typos and is the reliable way.

The @W:TeamNameForFile@ portion is replaced by the currently selected value:

- With TokyoTeam selected → Images/Teams/TokyoTeamLogo.png

- Switch to OsakaTeam → Images/Teams/OsakaTeamLogo.png

Switching the value via the Widgets panel dropdown or Prev / Next swaps the displayed logo automatically.

Check: Switching the Widget value swaps the logo automatically.

Tip: The Widget depends on the value type: text (team names etc.) = Text List, numeric counts = Counter, time = Clock Timer. The trick is unified file names of 'value + fixed word' (TokyoTeamLogo.png / OsakaTeamLogo.png...). To switch via external data, use %DataLinqKeyName% instead of @W:Name@.

STEP 6 Check before the show

Confirm every expected image resolves and displays. Three checks in particular:

- Do relative paths work on another PC (the playout machine)? Watch for different save locations or drive mappings
- Does changing the Widget/data value really switch the image?
- When a specified image is missing (empty/gap), does the layout avoid major breakage?

Actually switching several patterns in rehearsal is reassuring.

Tip: Most 'image missing on the production machine' cases are path-resolution mistakes. Bundle assets in the project and use relative paths to reduce accidents. Where asset locations change per venue, use file paths with caution.

Pre-show checklist

- ☐ With relative paths, images display on another PC (the playout machine)
- ☐ Changing just the Widget (@W:name@) or DataLinq key (%keyname%) value switches the image
- ☐ Filenames are unified as "value + fixed word" (e.g. TokyoTeamLogo.png)
- ☐ If using absolute paths, confirmed the same path resolves on the playout machine
- ☐ Missing images (empty/gap) don't badly break the display
- ☐ Matchup/lineup swaps run without manual work

4. Build a Simple Game Clock

Countdown display with a Clock Timer Widget

Vol. 4, Ch. 14 | Level: Intermediate | approx. 30 min | Folder: 04_GameClock

Goal

A simple game clock counting down from 12:00 to 00:00. The Text Object is the display; the time value lives in the Clock Timer Widget. Start / Stop / Reset work reliably in Manual, and Online / re-Take cause no unintended Reset or re-Start.

STEP 1 Split the approaches into three

Clock/timer displays need different tools for different goals.

- ① Simple countdown → Clock Timer Widget (this time).
- ② Staging tied to remaining time (e.g. color changes) → Visual Logic.
- ③ Displaying an official clock from an external scoreboard → DataLinq.

Here we build with the clearest option, the Clock Timer Widget. Don't force Visual Logic from the start.

Check: You can state "this build uses the Clock Timer Widget".

STEP 2 Create a test Scene

- ① Create a new Scene in the Scene Manager with a clear name like GameClock_Test
- ② No backgrounds, decorations, logos, or scores yet. What we verify first isn't looks but "does the clock display correctly and Start / Stop / Reset?"

Check: An empty Scene named GameClock_Test exists.

STEP 3 Place the display Text Object

- ① Drag one Text from the Object Library into the Scene and type a placeholder "12:00" (to check position, size, Font, Material)
- ② Name it by role, e.g. Clock_Text

Important: typing the placeholder does not make a running timer. The Text Object isn't the part that creates time — it's the display for the time value.

Check: Clock_Text shows the placeholder "12:00" (it doesn't need to run yet).

STEP 4 Create the Clock Timer Widget

Open Display > Widgets and create a Clock Timer via New Widget. Give it a purpose-revealing name like GameClock_Timer (it pays off when selecting from the Text Object later). The Widgets screen is both where you create Timers and where you monitor and operate them.

Check: GameClock_Timer appears in the Widgets panel.

Tip: As Widgets multiply you lose track of which Timer feeds which Text. Give purpose-revealing names from the start.

STEP 5 Set Mode to Timer

In the Clock Timer Widget's Properties set Mode to Timer. Clock = deals with the current time of day; Timer = advances/returns from an arbitrary start value. We want 12:00 down to 00:00, so Timer is the one.

Check: Mode is Timer.

STEP 6 Decide Start At / Stop At / Direction

For a 12-minute countdown: Start At = 12:00, Stop At = 00:00, Direction = Down. What to internalize isn't the setting names but the three ideas: where it starts, where it stops, up or down. Input formats and item names vary by version/screen — verify on the machine.

Check: Start At = 12:00, Stop At = 00:00, Direction = Down.

Tip: Get these three wrong and, even with the right display format, the timer becomes hard to use.

STEP 7 Decide the Format (start with NN:SS)

Format decides the string the Timer value becomes: NN:SS / HH:NN:SS / NN:SS.ZZZ etc. For a simple game clock, NN:SS (e.g. 12:00 / 05:30 / 00:10) is clearest. Decimals (.ZZZ) add digits and can jitter if the Font's digits aren't monospaced, so NN:SS is the safe start.

Check: The display is in NN:SS format.

STEP 8 Set Start / Stop / Reset to Manual

For the first build set the Method for Start / Stop / Reset to Manual (the operator acts explicitly). When Online makes it run the moment it's taken, isolating causes gets hard. Note: setting Manual and actually pressing Start are different things — Start comes later from the Widget side. If assigning shortcuts like Ctrl+1–9, verify they don't clash with Keyboard Mapping.

Check: Start / Stop / Reset Methods are all Manual.

STEP 9 Connect the Widget to the Text Object's Data Source

Select Clock_Text and open the Text Object's Data Source tab.

Choose Widget as the Data Source and pick GameClock_Timer from the list. Clock_Text now shows the timer value.

Don't just leave a typed "12:00" (that's a static display).

If nothing shows, check in order:

- Is the right Text selected?
- Did you choose Widget in Data Source?
- Did you pick the right Widget (GameClock_Timer)?
- Is the Format as intended?
- Any Font / Material / position issues?

Check: Clock_Text displays the Widget's value (12:00).

STEP 10 Verify the display in the Main Viewport

Once connected, confirm the value shows in the Main Viewport. What you're checking isn't design: is the Widget's value displayed / is the Format right / does the text fit? If motion isn't visible, also check display-refresh states like "Show or Hide Continuous Animations and Other Effects" at the top of the Viewport. Note: displayed ≠ started.

Check: Value and Format look right in the Main Viewport.

STEP 11 Manual Start → Stop → Reset from the Widget side

Start from the Widget side; if it counts 12:00 → 11:59 → 11:58, the display connection works.

If it doesn't move, check these before touching the Text Object's design:

- Has the Widget been started?
- Is Method set to Manual?
- Did you actually run Start?
- Are Start At / Direction as intended?

Then confirm Stop holds and Reset returns to the start value.

Reset is powerful (a mid-show mistake can jump the value back). Confirm who may Reset and when, and whether any auto-Reset (Reset on timer, etc.) is enabled.

Check: Start counts down, Stop holds, Reset returns to 12:00.

STEP 12 Go Online and verify Take/Offline/re-Take

Once Layout checks out, put the Scene Online and watch playout behavior. Working in the Main Viewport doesn't guarantee the same on air.

Confirm:

- Is the clock stopped when it goes Online?
- Does it stay stopped until Manual Start?
- On Offline: stop, hold, or Reset — which?
- On re-Take: any unintended Reset or re-Start?

Value retention/Reset on re-Take depends on Scene attributes and Sequencer/Widget settings — always verify on the machine so the operator can explain it.

Pre-show checklist

- ☐ Of the three approaches, you chose simple = Clock Timer Widget
- ☐ Built with the Text Object as display and the time value in the Clock Timer Widget
- ☐ Start At / Stop At / Direction are as intended (from where, stops where, up or down)
- ☐ Not conflating the Manual setting with actually pressing Start (displayed ≠ started)
- ☐ Confirmed Reset timing and authority, and any auto-Reset
- ☐ Confirmed no unintended Reset/re-Start on Online, Offline, or re-Take

5. Feed Scores and Names from CSV / Excel

Reducing manual entry with DataLinq and Widgets

Vol. 5, Ch. 3-4 | Level: Intermediate | approx. 35 min | Folder: 05_DataLinq_Practice

Goal

Feed player names, scores, and team info from an external table (CSV/Excel) into Text Objects, so editing the table changes the display. Choose the data source by who edits it and how it breaks, and anticipate what failure looks like.

STEP 1 Grasp DataLinq's common structure

DataLinq's common structure: ingest external data → reference by key (column/field) → display on a Text Object or Material. The skeleton is the same across CSV/Excel/JSON. First decide what value, by which key, goes where.

Check: You can say "what value, by which key, goes where".

STEP 2 Get CSV working first

- ① In XPression DataLinq Server, Add DataLinq Source → Text DataLinq Source (this is the one for CSV = delimited text; confirmed present in the list)
- ② Point File Path at teams.csv and confirm Encoding (UTF-8), delimiter, and header handling
- ③ On the XPression side: Text Object's Data Source tab → choose DataLinq → Set → pick the Source, Browse → choose the cell to display from the table (Column = column number / Row = row number)

Display one item first and check for mojibake or column shifts.

Check: The first CSV item's team name (Row=2) shows in the Text Object.

Tip: CSV is least troublesome with single-byte alphanumeric column names and UTF-8. The first (header) row becomes the keys.

STEP 3 Switch rows to change the display

Switching which row displays swaps names and scores.

[Machine-verified row numbering] For CSV, in Set DataLinq Properties Row = row number, Column = column number. The first line counts as the header row (TeamName etc.), so the first data item is Row=2 (e.g. Row=2 is TokyoTeam, Row=3 is OsakaTeam). Note this differs from JSON's players<2> (no header concept).

For a lower third assign one item at a time; for a scoreboard assign Home/Away to two different rows. Edit the CSV, reload, and confirm the display follows.

Check: Changing Row 2→3 switches to the next team (person). Editing and reloading the CSV updates the display.

Tip: To make row switching easier in operation, the same DataLinq Key method as the JSON hands-on (%KeyName% in the Row field) works here too.

STEP 4 Excel if editability matters, JSON if structured

If on-site staff frequently hand-edit, Excel is the likely pick (but verify on the machine which sheet/range/save format gets read). For nested fields or external system integration, JSON. Choose by what needs checking on site, not by dogma.

Check: You can explain which you'd pick for your site, with reasons.

STEP 5 Decide live behavior (updates and failures)

Live Update is handy but not for every item. A shorter Poll every isn't automatically better. Return Empty on Failure decides what shows when data can't be fetched. Anticipate whether a failure leaves the previous value or goes blank, and choose.

Check: Breaking the data (e.g. renaming the file) shows the expected result (blank / previous value).

Tip: Always build and check 'how it looks with empty data' once. If you use Prepend/Append, watch their behavior with empty data.

STEP 6 Add switching with Widgets (optional)

Widgets finish the job of reducing manual entry:

① Open Display > Widgets and create via New Widget: Text List (switch choices) / Counter (score up/down) / Clock Timer (time)

② A Widget alone shows nothing. It only takes effect once assigned via the Text Object's Data Source tab → Widget

Keep DataLinq (external data) and Widgets (operator-controlled values) conceptually separate.

Check: The Widget's value shows in the Text Object and changes when switched.

STEP 7 Check before the show

Choose the data source by who edits it and how it breaks. Confirm: does every expected dataset display correctly / do long rows avoid breakage / do missing, empty, or gap data avoid major breakage / can the staff follow the editing rules (don't delete columns, etc.)?

Pre-show checklist

- ☐ Decided "what value, by which key, goes where" first
- ☐ Confirmed one CSV item displays, then row switching swaps it
- ☐ Chose the format (CSV/Excel/JSON) by editing frequency, structure, and integration
- ☐ Decided Live Update / Poll every / Return Empty on Failure behavior
- ☐ Confirmed how empty data and gaps look
- ☐ If using Widgets, assigned them via Data Source so they show

6. Headline Reveals One Character at a Time with Stagger

Text staging that stays readable on air

Vol. 3, Ch. 2 | Level: Intermediate | approx. 25 min | Folder: 06_Stagger_Text

Goal

A headline whose characters enter slightly offset. Three steps — create the settings → place onto the target in the Scene Director → play and check — designing per-character/word/line offsets with Timing Offsets. The staging must survive text swaps.

STEP 1 Grasp the three-step flow

Stagger Animation is ① create the animation settings ② place those settings onto the target's row in the Scene Director ③ play and check how it looks.

Settings are made in the right-dock Stagger Animations panel (New... / Edit...) (machine-verified).

Check: You can recite "① create → ② place → ③ play".

STEP 2 Decide the target and names

Place the Text Object to animate. Creating an object automatically gives it a row (track) in the Scene Director — no manual adding needed (machine-verified). What is manual is placing your Stagger Animation onto the target Text's row.

Use the Stagger Animation's Name and Description for operations (so it's clear later what the motion is). In live graphics, being understandable later matters as much as looking good.

Check: The Scene Director has the target Text's row, and the Stagger Animation's Name/Description reveal its purpose.

STEP 3 Decide what moves with Tracks

① In the Stagger Animations panel: New... → select the created StaggerAnimation1 → Edit...

② In Edit Stagger Animation's Tracks, add the properties to animate (verified example: Position.X; for entrances, Alpha + Position is the classic)

Give Tracks only to the properties you'll move.

Check: Tracks contains only the intended properties (e.g. Position.X).

STEP 4 Shape the motion with Keyframes

- ① In the same Edit Stagger Animation's Track Animation, build the change curve (e.g. Position.X from off-screen to 0)
- ② Total Duration's Frames is the overall length (verified example: 150). Think of one character's change length (Clip Length) separately from the overall length
After changing the length, use Recalculate Keyframe Positions and confirm nothing shifted.

Check: Playing moves everything as one block (no offsets yet is fine).

Tip: Track Controls exist to reduce mid-work accidents (like moving the wrong Track by mistake).

STEP 5 Create the offsets with Timing Offsets (the heart of it)

- ① In the same dialog's Timing Offsets, enter the offset frames (verified example: Character 3.00 / Word 6.00 / Line 15.00 / Paragraph 30.00)
- ② Pivots' Mode (verified example: Baseline Center) is the center used by Scale and Rotation
- ③ Drag the finished Stagger Animation onto the target Text's row in the Scene Director
For Japanese headlines, Character or Word offsets usually read best.

Check: Characters appear in sequence per Character/Word.

STEP 6 Play and confirm readability

- ① Preview via Scene Director playback (or dragging the playhead)
- ② Judge not just "does it feel good" but "can it be read". Too fast and the eye can't follow
- ③ Swap the text via Template Links and confirm long and short names both hold up

Check: Text is readable mid-playback and survives swaps.

Pre-show checklist

- ☐ Proceeded in three steps: create settings → place on target → play and check
- ☐ Gave Name / Description that make sense later
- ☐ Gave Tracks only to what needs moving (Alpha/Position etc.)
- ☐ Kept Clip Length and Total Duration separate; recalculated after length changes
- ☐ Chose the Timing Offsets unit (Character/Word/Line/Paragraph)
- ☐ Confirmed readable speed and no breakage across text lengths

7. Design Ins and Outs with Transition

Scenes that leave correctly and update correctly

Vol. 3, Ch. 8 | Level: Intermediate | approx. 25 min | Folder: 07_Transition

Goal

Design not just the Scene's entrance (In) but its exit (Out) and updates (back-to-back), so how things linger is never unpredictable on air. Account for Layers, stacking order, and removal via external control.

STEP 1 Understand what Transition designs

Transition isn't a mere effect — it's the design item that keeps Scene ins and outs from breaking. It covers In (appearing), Out (leaving), and updates (content changing back-to-back).

Check: You can name In / Out / update as the three design targets.

STEP 2 Build In and Out first

Two places to set it (machine-verified):

- ① The Sequencer's Transition In / Out columns — set In and Out effects (Cut / Dissolve / Push etc.) per Take Item. Default is Cut / Cut
- ② For elaborate ins/outs, build them as In/Out moves in the Scene Director (for conditional use, the Object Inspector's Transition Logic tab)

Always set both In and Out first (without a designed Out, how things linger on air is unpredictable).

Check: Both In and Out are set; show-then-remove works end to end.

STEP 3 Decide Duration by on-air tempo

- ① Adjust lengths in the Sequencer's Transition In/Out settings
- ② The criterion is on-air tempo, not visual taste: persistent straps = natural Out / emergency displays = a path to instant (Cut) removal / staging Scenes = design through the Out / score updates = short enough not to block the next info

Check: Duration fits the purpose (persistent / emergency / staging).

STEP 4 Design updates (back-to-back)

Decide how continuous updates of the same Scene look (scores, info updates):

- ① Swap the same Take Item's values and re-TAKE; rehearse how back-to-back playout looks
- ② Decide and standardize: "full In/Out every time" or "swap contents only"

Check: Continuous updates look as intended.

STEP 5 Separate Scene Director and Transition Logic

Split the roles: Scene Director = where motion content is built / Transition Logic (Object Inspector - Scene tab) = where conditions pick which Scene Director runs.

If simple In/Out suffices, stick to the Sequencer columns and keep complexity to what's needed (overbuilt logic becomes unreadable data).

Check: You can explain which Scene Director runs when.

Tip: Assume pre-show fixes, different operators, duplication, Layer changes, and external-control changes — don't make it more sophisticated than needed.

STEP 6 Look at Layers, Framebuffers, and external control

Look beyond motion to stacking order (Layer / Framebuffer). To react to another Layer's state, consider Scene Triggers. If external control (GPI / RossTalk / PBus etc.) removes the Scene, confirm the actual removal matches the Transition's assumptions.

Check: Stacking order and externally-triggered removal behave as expected.

STEP 7 Pre-show check order

Don't check only the In — include updates, Out, Layers, and external control. Confirm: does it leave correctly after showing / do continuous updates hold / is stacking order right / does external removal behave as expected?

Pre-show checklist

- ☐ Built both In and Out (the Out is designed)
- ☐ Decided Duration by on-air tempo and purpose
- ☐ Designed back-to-back if continuous updates exist
- ☐ Kept Scene Director and Transition Logic roles separate
- ☐ Checked Layer/Framebuffer stacking order
- ☐ Confirmed externally-controlled removal matches expectations

8. Build a Staff Roll (Roll)

Designing Roll/Crawl with a Scene Group

Vol. 3, Ch. 9–10 | Level: Intermediate | approx. 30 min | Folder: 08_StaffRoll

Goal

An end roll where staff names and titles scroll vertically. Decide how it will be fixed and operated before how it moves, keep a safe visible area with Global Margins, and design how the Loop stops.

STEP 1 Decide the purpose first

Before creating a Scene Group, decide what flows and how. For a staff roll, group the Scenes showing names and titles. For a sponsor Roll, group the logo Scenes. Unglamorous, but this matters most. Think how it will be fixed (pre-show edits, other staff, reuse) before how it moves.

Check: What flows, in what order, is decided.

STEP 2 Choose Scene Group or Sequence

Scene Group = design cohesion first, managed close to the template structure. Sequence side (Take Item Group) = suits frequent content changes, managed in layout order. We use a Scene Group this time. Either way, build a structure the next person can understand.

Check: You can state why you chose the Scene Group.

STEP 3 Create the Scene Group and stack "pages" inside

The roll's substance is child Scenes stacked inside a Scene Group (machine-verified structure).

- ① Create a Scene Group in the Scene Manager with a name that reads later, like EndRoll_Staff
- ② Add child Scenes inside the group. One child Scene = one page of the roll. They flow in the top-down order shown in the Scene Manager
(e.g. page 1 = title page, page 2 = staff list, page 3 = sponsors/closing page)
- ③ Build each page like a normal Scene (place Text with names and titles). One page's height = the Scene height, so split long staff lists across pages
- ④ Settings are centralized on the group: select the Scene Group → the Object Inspector's "Scene Group" tab is the entrance to Roll conversion, speed, margins, and looping

Check: Child Scenes hang under EndRoll_Staff in the Scene Manager, ordered as intended.

Tip: Vague group names mean nobody can touch it at fix or reuse time. Page order is decided by the order in the Scene Manager.

STEP 4 Choose Roll (vertical) or Crawl (horizontal) with Effect

Select the Scene Group and set it in the Object Inspector's Scene Group tab → Group section (machine-verified):

- Effect: Roll (vertical) / Crawl (horizontal). A staff roll is Roll
- Direction: Bottom To Top (following the eye)

Check: Effect = Roll, Direction = Bottom To Top.

STEP 5 Decide Duration (speed / time feel)

Set it in the Duration section of the Scene Group tab from step 3 (machine-verified: radio choice of Speed / Seconds / Frames):

- ① First shape the overall feel with Speed (verified example 2.000)
- ② Use Seconds to manage by duration; fine-tune with Frames

Too fast can't be read; too slow drags. Readability first.

Check: The speed lets you read to the end.

Tip: Rough it in with Speed/Seconds first, then fine-tune with Frames.

STEP 6 Keep a safe area with Global Margins

In the same Scene Group tab's Global Margins section (Top / Bottom / Left / Right), create a range that stays safe against the screen edges.

An operational setting so straps don't clip or bleed at the edges.

Check: No text clips at the edges; everything stays in the safe range.

STEP 7 Design leading/trailing blanks and the Loop

Set in the same Scene Group tab (machine-verified layout):

- ① Header / Footer section: check Blank Page on Start / Blank Page on End (prevents text starting abruptly or cutting off suddenly)
- ② Loop section: Enable Looping and Number of shows per scene. Loop is handy, but decide how to stop it live before using it
- ③ Start / Stop section: judge Ease In / Out (Frames) by readability, not by feel

Check: Start and end aren't abrupt, and the Loop's stop is decided.

STEP 8 Check before the show

Confirm: readable speed to the end / nothing clips at edges and stays in the safe range / start and end aren't abrupt / if looping, it can be stopped as intended / Per Scene Lighting etc. doesn't change the look. Also confirm swaps (added names) don't break it.

Pre-show checklist

- ☐ Decided how to fix/operate before how to move
- ☐ Chose Scene Group vs. Sequence and can explain why
- ☐ Used names that read later
- ☐ Speed prioritizes readability (Speed/Seconds → Frames)
- ☐ Global Margins prevent edge clipping
- ☐ Designed the Loop's stop and the leading/trailing blanks

9. Create a Visible Area with Mask

Designing what to hide, cut out, and show

Vol. 2, Ch. 14 | Level: Intermediate | approx. 25 min | Folder: 09_Mask

Goal

Create a rectangular display area (wipe / cutout) and make it movable. Think "decide what's visible" rather than "erase", and Publish only the values that need live adjustment so operations don't break.

STEP 1 Grasp the Mask mindset

A Mask decides what's shown, it doesn't erase. Decide what to show and what to hide before building and you won't get lost. Before using it, confirm which area, when, and how it should show.

Check: You can put the visible and hidden areas into words.

STEP 2 Group the targets with a Layer Object

- ① Add Layers > Layer Object from the Object Library to the Scene
- ② Drag the objects to be masked so they become children of the Layer Object in the Object Manager

The Layer delimits where the Mask applies (confirmed in official help: mask only within one Layer without affecting other objects or Layers).

Check: Target objects are gathered under the Layer Object.

STEP 3 Create the rectangular area with Box Mask

- ① Add a Box Mask object to the Scene and place it inside the Layer (added from the Object Library; check the label in your environment)
 - ② Inside the rectangle = visible. Adjust the area's position and size via Transform (Position / Scale)
- To cut out with an image shape, use the Mask Material + alpha asset approach.

Check: Only the inside of the Box Mask is visible.

Tip: First get a solid rectangular region with a Box Mask, then move on to alpha-asset cutouts if needed.

STEP 4 Animate the Mask in the Scene Director

- ① On the Box Mask's row in the Scene Director, build a Position (or Scale) animation (e.g. a wipe-in expanding left to right)
- ② Play and confirm the reveal/conceal motion

Check: Playback moves the area and it reads as a wipe.

STEP 5 Publish only the values that need adjustment

- ① Check only the values adjusted during the show on the target object's Template Links tab to publish them
- ② Published values appear in the Sequencer's Take Inspector. Name them so operators understand them

Criteria: will it really be adjusted live / any leftover build-time values / any values published that must not be touched / can defaults be restored

Check: Only published values appear in the Take Inspector, with understandable names.

STEP 6 Pre-show checks and isolation

Confirm: only the intended area shows and the rest hides / moving the area doesn't break anything / operators understand the published values. When it misbehaves, isolate in this order: Layer structure → Mask type → stacking order.

Pre-show checklist

- ☐ Designed as "decide the visible area", not "erase"
- ☐ Grouped the targets with a Layer Object
- ☐ Built the rectangular display area with Box Mask
- ☐ The area can be animated in the Scene Director
- ☐ Published only live-adjusted values; hid untouchable ones
- ☐ Defaults are restorable / item names make sense to operators

10. Build Readable Color Fields with Gradient

Designing usable color fields, not decoration

Vol. 2, Ch. 10 | Level: Beginner | approx. 20 min | Folder: 10_Gradient

Goal

Build a "reading cushion" that slightly darkens the area under text, or a panel fading to transparent at the edge to blend with video. Design controllable color fields from the purpose, not for flash.

STEP 1 Decide the purpose first

Gradient is a four-color linear gradient, opacity included. Decide the purpose before building: brighter top and darker bottom / a dark cushion only behind text for readability / fading to transparent toward the edge to blend into video, etc. Don't think of it as just a pretty background.

Check: You can state this color field's purpose (blending / text cushion, etc.).

STEP 2 Place the information first

Before the colors, decide where text and numbers go. Place placeholder text first, then build the color field, and it won't turn unreadable later. Colors made before the layout tend to break once text lands.

Check: Placeholder text is placed and its position decided.

STEP 3 Give the four colors roles

- ① Create a new Material in the Material Editor and choose Add Shader > Gradient (menu confirmed to exist)
- ② Set each of the four corner colors with a role: two top colors bright, two bottom dark (variation in one direction only) / darken one side as a text area, etc.

The aim is giving brightness and hue changes a role, not making it flashy.

Check: You can explain each corner's role.

STEP 4 Balance "blending" and "reading" with opacity

- ① Adjust each corner color's alpha: lower only the outer colors to blend into video / keep the text area near-opaque
- ② You can also lower just one corner's opacity for an airy touch

Check: Edges blend, and the text area stays dense enough to read.

Tip: Opacity is the balance between blending and readability. Keep what sits under text dark enough to read.

STEP 5 Change little by little and verify

Adjust in small steps. Controllability beats flash. Lower thirds: darker under the text. Score displays: dense enough that digits read. Team colors: blended, not overpowering.

Check: It reads with actual text on it.

Pre-show checklist

- ☐ Decided the purpose (read / blend) before building
- ☐ Placed the text/number areas before the colors
- ☐ Gave the four colors roles (constrained the direction of change)
- ☐ Opacity balances blending and readability
- ☐ The area under text is dense enough to read
- ☐ The result is a controllable color field, not a flashy one

11. Show the Current Time and a Countdown

Treating time as changing information with Date Time

Vol. 4, Ch. 5 | Level: Intermediate | approx. 30 min | Folder: 11_Clock_Countdown

Goal

Build a current-time display and a countdown to a game/event start. Treat time as changing information rather than text: build a target time, compute the difference from now, and format it into a readable string.

STEP 1 See time as changing information

Practical time displays (start time, start date, countdowns, weekday, switching at a set time) assume time changes. Build a mechanism that holds as values change — not a finished still image.

Check: You can explain that this display is "changing information".

STEP 2 Clock or Date Time blocks — decide which

If you just show the current time or date, Clock suffices. For "difference from a target time (countdown)", "decomposing a datetime to get the weekday", or "building a datetime from a given date", you need the Date Time blocks.

Both live in the Visual Logic Editor under Function Blocks > Date Time (confirmed in the machine's block list).

Check: You decided whether Clock is enough or Date Time blocks are needed.

STEP 3 Show the current time as-is (Clock)

- ① Open the Scene's Visual Logic Editor
- ② Drag Clock from Function Blocks > Date Time
- ③ Clock output → Format Date Time (shape into a display string) → the display Text's Text
- ④ Use a format showing only the digits you need (hours/minutes/seconds)

Check: The current time shows in the Text and the seconds advance.

STEP 4 Build the target time (Encode Date Time)

- ① Drag Encode Date Time (Date Time)
- ② Enter year/month/day/hour/minute/second (fed by Value blocks from Math) to assemble the target datetime, e.g. the game start

Check: Encode Date Time outputs the target datetime.

Tip: Check that boundary cases like end-of-month ± 1 month (e.g. one month after 1/31) behave as expected on the real system.

STEP 5 Compute the difference for time remaining (Time Delta)

- ① Drag Time Delta (Date Time)
- ② Wire Clock (now) and Encode Date Time (target) into Time Delta to compute the difference
- ③ Use the Days / Hours / Minutes / Seconds outputs to display the remaining time

Check: The remaining time (target — now) displays and decreases each second.

STEP 6 Shape it for display (Format Date Time)

- ① Shape the display string with Format Date Time. If you need the weekday, extract it with Day of the Week
- ② Width moves as digit counts change, so check the Text Object's width and alignment (the Font's Monospace Numbers setting stabilizes digits)

Check: The format is as intended and width holds as digits change.

STEP 7 Check before the show

Confirm: the current time updates correctly / the display holds as the countdown nears zero / alignment holds as digit counts change / time zone and reference time are as intended. Since values keep changing, leave it running for a while and observe.

Pre-show checklist

- ☐ Treating time as changing information
- ☐ Decided whether Clock suffices or Date Time blocks are needed
- ☐ Built the target time with Encode Date Time
- ☐ Computed the difference with Time Delta and displayed the remaining time
- ☐ Shaped it with Format Date Time and checked digit jitter
- ☐ Ran it a while and confirmed no update/boundary breakage

12. Build a Virtual Monitor with Render View

Turning a Scene into an asset inside your staging

Vol. 2, Ch. 4 | Level: Intermediate | approx. 30 min | Folder: no materials needed (built with features only)

Goal

Paste an info display built in another Scene (score or player card) onto a Quad — a virtual monitor — inside a staging Scene. Experience "Scene as a component": fixing the source Scene updates the staging side too.

STEP 1 Prepare the source info Scene

Build the information to show (e.g. a score panel or player card) as an independent Scene. Name it so it reads as a component, like SRC_ScorePanel. Build at a readable size, with a transparent or solid background for easy pasting.

Check: The source Scene displays correctly on its own. Don't give it the same name as the staging Scene (same names hide which is the source; e.g. source = SRC_ScorePanel / staging = RV_Monitor).

STEP 2 Place the target Quad in the staging Scene

- ① Drag Primitives > Quad from the Object Library into the staging Scene
- ② Name it by role in the Object Manager, e.g. Monitor_Main
- ③ A slight angle makes it look like a monitor within the picture

Check: The Quad sits at the intended position and angle.

STEP 3 Add a Render View Shader to the Material

Create a new Material in the Material Editor and choose Add Shader > Render View. Specify the source Scene and camera in Scene / Camera Source, then assign this Material to the Quad.

Check: The source Scene's picture appears on the Quad's face.

Tip: When nothing shows, check in order: the source Scene assignment → the camera assignment → where the Material is applied. Choosing an Ortho Camera on the source side keeps panel text undistorted — a 'screen meant to be read' (Vol. 3's camera choice in practice).

STEP 4 Decide Resolution from the displayed size

Decide the Render View's Resolution (Width / Height) from how large it finally appears on screen. Machine-tested reference (a 1920×1080 panel pasted onto a large LED): 100×100 = clearly blurred, unreadable / 480×270 = a bit soft / 1920×1080 = sharp. Too high costs render load; too low blurs text.

Check: Drop it to 100×100 once to feel the blur, then return to a readable resolution.

STEP 5 Adjust the fit with Texture Coordinates

Tidy the pasted picture's position and scale with Texture Coordinates, adjusting overflow and margins against the panel.

Check: The source's edges sit correctly within the Quad.

STEP 6 Verify updates and the reference graph

Change text or colors in the source Scene and confirm the staging Scene's virtual monitor updates. [Machine-verified] Renaming the source Scene keeps the Render View Material's Scene reference intact (an internal reference, not the name string — renames don't break it). Deletion is different: always check usages before deleting a source Scene.

Finally, be able to explain which Scene is used by which Material on which Object.

Check: Source-side changes appear on the virtual monitor. Renaming keeps the display.

Tip: Heavy use of Render View complicates the reference graph. Use it only where you can state the reason for making something a component.

Pre-show checklist

- ☐ The source Scene and Material have component-revealing names
- ☐ Resolution matches the displayed size (no excess, no shortfall)
- ☐ Text is readable in the pasted state
- ☐ The staging Scene's overall load is acceptable
- ☐ Live-adjusted values are separated from build-time fixed ones
- ☐ You know the impact range of deleting or renaming the source Scene

13. Cycle a Player List with JSON

Treating hierarchical data as tables with Table Presets

Vol. 5, Ch. 3 | Level: Intermediate | approx. 40 min | Folder: 13_JSON_PlayerList

Goal

Load a player-list JSON via DataLinq and display player name, number, and position in Text. Then use a DataLinq Key so changing one key switches the whole row (name, number, position) together — the real operational pattern. Uses the players.json set in the 13_JSON_PlayerList folder.

STEP 1 Prepare the sample JSON

Use players.json from the 13_JSON_PlayerList assets (UTF-8, no BOM). Structure: team > players[] > name / name_en / number / position. Player 6 deliberately has an extra-long name for width-breakage checks.

Check: You can open it in an editor and explain the structure (player objects inside a players array).

STEP 2 Add a JSON Source in DataLinq Server

In XPression DataLinq Server (a separate window from XPression), Add DataLinq Source → choose "JSON DataLinq Source" (Provides access to JSON data files) → OK.

In JSON Linq - Configuration:

- File Path: point at players.json
- Encoding: ASCII (default) → must change to UTF-8 (ASCII garbles Japanese)
- Update method: for local files Wait for file change events (per the on-screen note, not recommended on network drives), or Poll every N seconds

Check: The Source adds without error and Encoding is UTF-8.

Tip: Turning on Enable Logging makes trouble easier to trace later.

STEP 3 Register "tables (files)" with Table Presets

Do this before assigning to Text (later row sync fails if the Table isn't set correctly, so it's effectively required). Table Presets don't specify a hierarchy inside the JSON; they register multiple JSON files by Name + File Path so the Table list can switch between them when browsing data (the settings screen says they appear in the Tables dropdown when browsing for data).

Steps:

- ① On the Settings tab check Enable Table Presets → open the Table Presets tab
- ② Add → name in the Name column, choose the JSON via "..." in File Path
- ③ For the bundled samples register these four (filename on the right; File Path receives the absolute path picked via "..." — don't count on relative paths):

Players_Player → players.json

Players_Short → players_short.json

Players_MissingName → players_missing_name.json

Players_Empty → players_empty.json

The trick is "common prefix (Players_) + variant name" — it makes Key-driven Table switching (step 5's extension) clean later.

- ④ OK to save → the four appear in the Table list of the Text's Browse (Select DataLinq Field); picking one switches the source file

Check: Switching the Table to Players_Empty blanks the display; back to Players_Player restores the players.

Tip: Entering only Name with an empty File Path does nothing. File Path becomes an absolute path, so it breaks when moved to another PC. In production, the standard practice is a fixed folder shared by all machines (e.g. C:\XPression_Data\) and registering that path (see Vol. 6, 'Surviving Delivery').

STEP 4 Assign to Text (pick the Table, then choose from the tree)

On the XPression side select the player-name Text Object, Object Inspector's Data Source tab → choose DataLinq → Set.

In Set DataLinq Properties check Enabled → pick this JSON source from the DataLinq list → Browse.

In Select DataLinq Field, first pick Players in the Table dropdown, then choose name inside players in the tree (the Selection at top shows Column: players / Row: name / Table: Players) → OK.

Assign number to the number Text and position to the position Text the same way.

Check: "佐藤 大輝", "1", "GK" display without mojibake and the Table field says Players.

Tip: If text is garbled, suspect step 2's Encoding (UTF-8?) first. A wrong Table breaks the later Key linkage, so verify it here.

STEP 5 Switch whole rows with a DataLinq Key (the heart — machine-verified)

Goal: change one key and the name, number, and position all switch to the same player's row together.

[Confirmed on the machine] Row addressing is angle brackets + 1-based row number (2nd player = players<2>). Value-search syntax (players<number=7> etc.) does NOT work. players<%PlayerNo%> is the correct form. It won't work unless the Table field is correctly set (Players).

- ① Select the Scene in the Object Manager → Object Inspector's DataLinq Keys tab → Add. Name: PlayerNo (check Published, initial Value 2).
- ② Open each Text's (name / number / position) Set DataLinq Properties and set the Column field to players<%PlayerNo%> (wrapped in %). Row and Table stay as-is.
- ③ Change the Key's Value and confirm. Timing: preview = immediate / an on-air Scene = after taking it Offline and re-TAKE (same iron rule as text swaps).

Check: Changing the Key 2→3 switches the preview immediately to 高橋 悠人 / 11 / FW in sync. On-air updates on re-TAKE.

Tip: Out-of-range row numbers (e.g. 99) behave per the Wrap Indices setting (ON = wraps to a valid row / OFF = treated as missing). [Advanced, machine-verified] Make the Table field Players_%TeamTable% and add a TeamTable DataLinq Key (values like Player / Short / Empty), and the referenced file also switches by Key. Combined with the row's PlayerNo, two Keys fully control 'which table, which row'.

STEP 6 Change the Key from the Sequencer and externally (RossTalk)

A Published Key appears in the Sequencer (Take Inspector's Template Data); the operator just types a row number to switch players.

Externally, RossTalk's DATALINQKEY command does the same.

Syntax: DATALINQKEY [takeid]:[keyname]:[value]

Machine-verified reliable sequence (switch Take ID 0002 to the row-3 player 高橋 悠人 and play out):

DATALINQKEY 0002:PlayerNo:3 ← ① swap the content to 高橋

CUE 0002 ← ② cue the Take Item with the new value (staging)

(wait ~200 ms) ← ③ wait for the cue to complete

TAKE 2 ← ④ put it on screen (playout)

TAKE immediately after changing the value can miss the update, so inserting CUE plus a short pause is the reliable way. The key name must exactly match the published name.

Check: Changing the Key in the Take Inspector switches the display. Sending the sequence externally plays out the new player reliably.

Tip: Key values are held per Take Item. Create several Take Items from the same Scene to give player A and player B separate row numbers (a standard broadcast setup for external-system integration). Stream Deck-style tools (Companion etc.) can put these 4 steps on one button.

STEP 7 Verify data updates

Edit values in players.json (names or numbers), save, and confirm the display updates. If not, revisit step 2's update method (Wait for file change events / Poll every).

Check: After saving, the display changes at the expected timing.

STEP 8 Watch behavior with broken data (Return Empty on Failure is the key)

Switch the Table to the presets registered in step 3 — Players_Short (fewer players) / Players_MissingName (a player without name) / Players_Empty (empty array) — and watch the display.

[Dangerous behavior confirmed on the machine] With each Text's Return Empty on Failure left OFF, when a row or field is missing, unintended values — like the number typed into the Key — appear on screen as-is.

Fix: in each DataLinq-linked Text's Set DataLinq Properties, check Return Empty on Failure (ON) → missing data shows blank.

Also sort out Wrap Indices: ON = out-of-range row numbers wrap back to valid rows (something always shows) / OFF = out-of-range counts as missing. If you must never show the wrong player, Wrap Indices OFF + Return Empty ON is the safe combination.

Check: With Return Empty on Failure ON, missing data shows blank instead of odd values.

Tip: Also check the layout doesn't break when fields are empty (an orphaned backing plate, etc.). Use Enable Logging / Data Logger to investigate.

Pre-show checklist

- ☐ Encoding is UTF-8 (not left at the ASCII default)
- ☐ The DataLinq Key name (PlayerNo etc.) is meaningful and Published
- ☐ Table Presets (Name + File Path) switching works in the Table list
- ☐ Checked the display with empty data and missing fields
- ☐ Verified the update method (file change / Poll) on the playout machine
- ☐ Agreed with the external system on rules for renaming fields or changing hierarchy

14. Take from Outside with RossTalk

A TCP taste of external control

Vol. 5, Ch. 6 | Level: Intermediate | approx. 30 min | Folder: 14_RossTalk

Goal

Enable Smart GPI / RossTalk over TCP and take Take Items on/off from outside with TAKE / SEQI / SEQO and friends. Experience the basics of external control and the differences in how things are removed.

STEP 1 Enable RossTalk in Hardware Setup

Open Edit → Hardware Setup, and on the "GPI / Tally Boards" tab, Add → choose Smart GPI / RossTalk from the Brand list (the list also shows PBus / Serial GPI (CTS/DSR) / TSL Tally). In Smart GPI / RossTalk Setup set State to Enabled, Mode to TCP, and note the TCP Port under Incoming Network Settings (e.g. 7788). When the list shows State Active, you're set.

Check: State=Active, Mode=TCP, port number noted.

Tip: On the production machine, also check Windows Firewall inbound permission. Keyboard / GPI Mapping lives in the same Edit menu.

STEP 2 Prepare the target Take Item

- ① Drag a test Scene into the Sequencer to make a Take Item
- ② Confirm/set the number in the Take ID column (e.g. 0002). Also confirm the destination framebuffer and layer
- ③ First confirm it plays correctly with a manual Take (numpad +) — manual must work before external control

Check: A manual Take (numpad +) plays it correctly.

STEP 3 Connect over TCP from outside (DashBoard is the practical way)

On the sending side, a CustomPanel in Ross's free DashBoard is practical: create a button, choose RossTalk as the Task → Connection=Custom, Host=XPression's IP address (localhost on the same PC), Port=the port from step 1. Command is a dropdown with TAKE / CUE / SEQI... ready — just fill in the Take ID.

You can also type commands directly from a terminal (PuTTY raw connection etc.; mind the CR/LF line endings).

Check: The button run (or connection) goes through without error.

Tip: DashBoard can insert Pause (wait) tasks, so a multi-command sequence like 'DATALINQKEY → CUE → Pause 200ms → TAKE' fits on one button. Open the verified sample panel in the 14_RossTalk folder — fix Host/Port and every button is ready to try.

STEP 4 Put it up with TAKE

Send "TAKE 1" or "TAKE 1:0:7" (TakeID:framebuffer:layer) and confirm it goes on air. TAKE's trait: the Sequencer's focus (selected row) doesn't move.

Check: The command puts the Scene up and the Sequencer selection stays put.

STEP 5 Try SEQI and SEQO

Send SEQI 1 (plays to the template-defined output; focus moves to that item) and SEQO 1 (takes that Take ID down), and compare with TAKE.

Check: SEQI moves the selected row; SEQO removes it.

STEP 6 Check the cleanup commands

Try LAYEROFF 0000:7 (takes the layer's Scene down with its Out transition), CLFB 0000 (clears the framebuffer), and CLRA (clears all framebuffers), and compare how they leave.

Check: LAYEROFF leaves with the Out motion; CLFB / CLRA clear instantly.

Tip: CLRA is a powerful clear-everything command. Decide its place in your show (including the option of never using it).

STEP 7 Call functions with the GPI command (optional — no physical gear needed)

RossTalk's "GPI 1" is a simulate command that fires a GPI input in software — no physical GPI hardware needed.

- ① In Edit → Keyboard / GPI Mapping, assign a function to simulated GPI input 1 (e.g. advance the Sequencer selection)
- ② Send GPI 1 from the DashBoard RossTalk task (or a terminal)

Check: The GPI command runs the assigned function.

Pre-show checklist

- ☐ Verified the port number and firewall on the playout machine
- ☐ The sender honors CR/LF command termination
- ☐ Take ID numbering rules (ranges with meaning) are decided
- ☐ Operators can explain the focus difference between TAKE and SEQI
- ☐ There's a procedure to notice a dropped connection mid-show
- ☐ Verified recovery from missends (SEQO / LAYEROFF)